

PyLocalSend 局域网文件传输系统技术报告

PyLocalSend 项目组

2026 年 6 月 12 日

摘要

PyLocalSend 是一个面向可信局域网的文件传输工具。发送端只登记本机文件或文件夹路径，接收端通过命令行或浏览器按授权下载；在被允许时，接收端也可以通过浏览器向发送端上传文件。项目采用 Python 实现，核心技术包括 FastAPI HTTP 服务、NiceGUI WebUI、SQLite 元数据持久化、HTTP Range 断点续传、分片流式传输以及可选 AES-CTR 加密。本文从系统目标、总体架构、传输流程、权限模型和测试情况五个方面概述项目技术实现。

1 项目背景与目标

在课堂、宿舍、实验室等局域网环境中，常见传文件方式都有额外成本：U 盘依赖硬件且多人复制效率低，网盘需要公网中转，聊天软件对大文件和目录结构支持有限。PyLocalSend 的目标不是构建完整云存储系统，而是在可信局域网中提供低步骤、可管理、能处理大文件的临时共享能力。

项目的核心取舍是“登记路径，按需读取”。发送端注册共享文件时，仅保存绝对路径、文件名、大小、修改时间等元数据；真正下载时才从磁盘分片读取并写入 HTTP 响应。这样避免把大文件预先复制到项目目录或一次性读入内存。

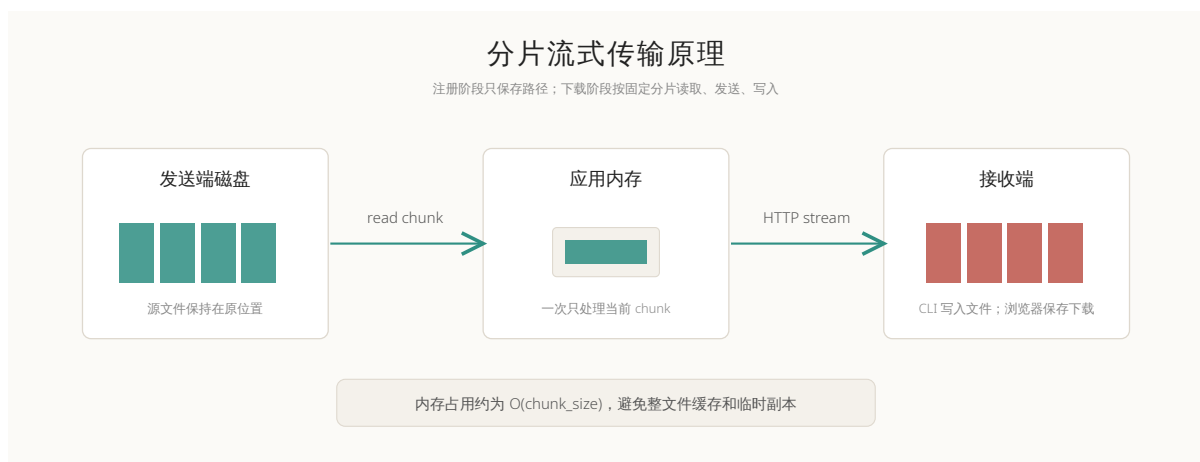


图 1: PyLocalSend 采用分片流式传输，内存占用主要由 chunk size 决定。

2 技术栈与目录结构

项目使用 Python 3.10+, 依赖集中在 `pyproject.toml` 中。FastAPI 和 Uvicorn 负责 HTTP API 与流式响应, NiceGUI 负责发送端和接收端 WebUI, httpx 负责 CLI 接收端请求, Rich 用于命令行进度展示, SQLite 保存元数据与日志, cryptography 提供 AES-CTR 加密能力。

目录或模块	职责
<code>pylocalsend/cli/</code>	命令行入口, 提供启动服务、注册共享项、远端列表、下载、配置和接收端管理命令。
<code>pylocalsend/core/transfer/</code>	发送端服务, 挂载 FastAPI 路由, 处理鉴权、注册路径、下载流、ZIP 流和接收端状态。
<code>pylocalsend/core/receiver/</code>	CLI 接收端客户端, 基于 httpx 获取文件列表、目录树并下载文件。
<code>pylocalsend/core/file_handler/</code>	文件元数据、目录递归、相对路径计算、分片读写和开放路径过滤。
<code>pylocalsend/core/utils/</code>	配置、数据库、加密、网络地址和日志工具。
<code>pylocalsend/gui/</code>	NiceGUI 发送端管理页和接收端浏览器页面。
<code>tests/</code>	覆盖下载、鉴权、接收端禁用、浏览器上传、网络工具和 CLI 行为。

3 系统架构

系统由一个发送端服务和多个接收端组成。发送端启动 FastAPI 应用, 同时通过 NiceGUI 提供管理界面; 接收端可以选择 CLI, 也可以打开发送端生成的 token 链接进入浏览器 WebUI。核心业务逻辑位于 `core` 层, CLI 和 GUI 都复用同一套注册、鉴权、传输和数据库接口。

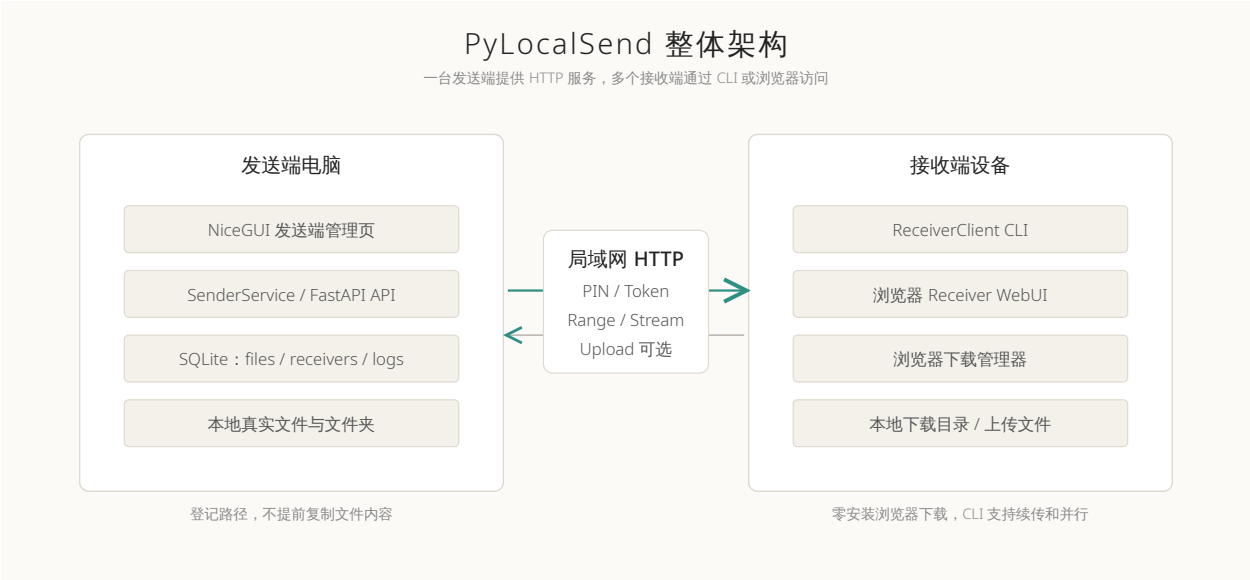


图 2: PyLocalSend 的发送端、局域网 HTTP 通道与接收端组件。

发送端持久化数据默认位于用户目录的 `~/.pylocalsend/`: `config.json` 保存端口、分片大小、PIN、上传目录等配置; `session.json` 可保存接收端连接信息; `pylocalsend.db` 保存共享项、接收端、下载日志和开放路径。

4 核心传输流程

文件下载流程可以分为六步：发送端登记路径，按 `download_grants` 过滤可见范围，接收端携带 PIN 或 token 发起请求，发送端根据 Range 头定位字节区间，按 `chunk_size` 分片读取并可选加密，最后通过 `StreamingResponse` 返回给 CLI 或浏览器。



图 3：从路径登记到分片流式返回的下载数据流。

4.1 分片与断点续传

`async_read_chunks` 根据起始偏移、长度和分片大小读取文件。CLI 下载前会检查目标文件是否已存在；若存在，就在请求头中设置 `Range: bytes=<existing>-`，发送端返回 206 Partial Content 和 Content-Range，接收端以追加模式继续写入。这一设计让中断后的大文件下载不必从头开始。

4.2 文件夹下载

文件夹有两种处理策略。CLI 路径先请求目录树，然后在接收端按相对路径递归下载并重建目录结构，适合批量、并行和可控保存位置。浏览器路径则请求 `/api/download/{file_id}/zip`，发送端边遍历文件夹边写 ZIP 响应流，不先生成临时压缩包。ZIP 使用 `allowZip64=True`，可覆盖较大的目录内容。

4.3 可选加密

当配置 `encryption_enabled=true` 时，发送端会用 PIN 派生 AES-256 密钥，并以文件 ID 和字节偏移生成 CTR 计数器。CTR 模式的好处是可按偏移独立加解密，因此与 Range 断点续传兼容。浏览器下载路径默认返回明文，因为浏览器原生下载管理器无法执行项目内的解密逻辑；CLI 路径则可按偏移解密后写入目标文件。

5 权限、接收端与状态控制

PyLocalSend 的访问控制不是复杂账号体系，而是由三类轻量机制叠加完成：统一 PIN 用于普通 API 鉴权；每个接收端可以拥有独立 token 链接；`download_grants` 记录每个共享项中真正对接收端开放的相对路径。空字符串表示整个共享项开放，具体子路径表示局部开放。

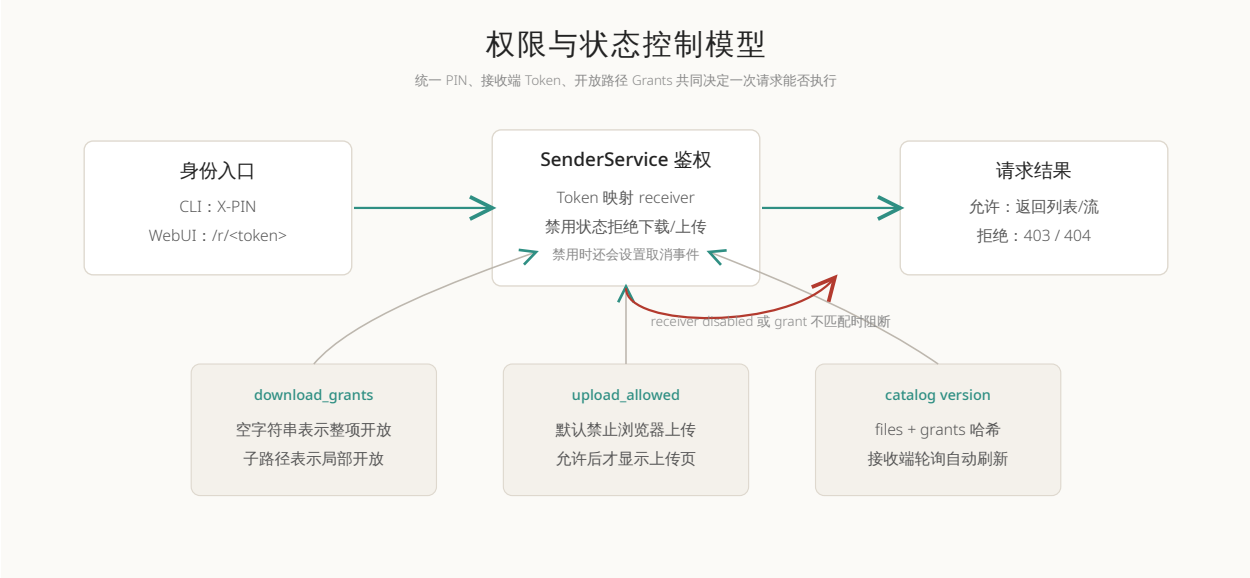


图 4: PIN、Token、Grants、上传开关和接收端状态共同决定访问结果。

接收端状态由 `receivers` 表管理。发送端可以禁用某个接收端，禁用后新下载和新上传都会被拒绝；若已有传输正在进行，服务端记录该下载对应的取消事件并在流式发送循环中终止。浏览器上传使用独立开关 `upload_allowed`，默认关闭；允许后，接收端 WebUI 才显示“上传文件”标签页。上传文件只写入 `upload_dir/接收端名称/`，不会自动加入共享列表，发送端仍需手动注册后才可再次对外开放。

为了让接收端 WebUI 自动同步，数据库会将 `files` 和 `download_grants` 的关键字段计算为短哈希，作为 `catalog version`。接收端约每 2 秒轮询 `/api/files/catalog-version`，版本变化时重新拉取列表和已展开目录树。

6 接口与数据持久化

系统主要 HTTP API 如表所示。CLI 与 WebUI 的差异主要在前端交互和本地保存方式上，真正的数据接口仍由同一个发送端服务提供。

API	用途
GET /api/files	返回可见共享项；token 访问时仅返回有开放路径的项目。
GET /api/files/{id}/tree	返回目录树；token 访问时按 grants 过滤。
GET /api/download/{id}	下载单文件或目录中的一个相对路径，支持 Range 和可选加密。
GET /api/download/{id}/zip	将目录按开放范围打包为 ZIP 流，供浏览器下载。
POST /api/files/browser-upload	接收端浏览器上传文件，要求 token 且已允许上传。
POST /api/receivers/{id}/disable	禁用接收端，并中断进行中的传输。
GET /api/files/catalog-version	返回列表版本，用于接收端 WebUI 自动刷新。

数据库 schema 保持简单：files 保存共享路径与元数据，receivers 保存接收端 token、PIN、状态和上传权限，download_logs 保存传输开始时间、结束时间、状态与字节数，download_grants 保存每个共享项的开放相对路径。该设计适合单机发送端场景，部署和备份成本低。

7 个人贡献：李子健（队长）

李子健主要负责项目的入口层、交互层和部分核心逻辑实现，同时承担队长的整体协调工作。在代码编写方面，李子健参与了 CLI 命令体系设计与实现，包括发送端启动、路径注册、远端列表、下载、配置管理和接收端管理等命令；参与发送端 NiceGUI 管理界面的实现，完成文件传输、接收端管理、设置页面以及 PIN 鉴权入口；参与接收端 WebUI 的文件选择、下载触发、上传标签页和自动同步交互实现；并对文件树勾选、开放范围展示、接收端状态反馈等前后端联动逻辑进行了调试。

在核心功能上，李子健参与了共享路径管理、下载开放范围、接收端 token 链接、浏览器上传开关等业务流程的联调，使 CLI、发送端 WebUI 和接收端 WebUI 可以复用同一套 core 层接口。测试与质量保障方面，李子健参与编写和维护 CLI 行为、GUI 鉴权、接收端禁用与浏览器上传相关测试，保证主要用户路径可重复验证。非技术工作方面，李子健负责项目统筹、演示流程设计、部分展示文档整理和短视频剪辑。

8 个人贡献：杨秉熹

杨秉熹主要负责项目传输层、数据层和权限控制相关代码，与李子健共同完成核心代码编写，整体技术贡献按约 1:1 统计。在代码编写方面，杨秉熹参与 SenderService 与 FastAPI 路由实现，完成文件列表、目录树、单文件下载、目录 ZIP 下载、接收端管理、配置读写等 API 的设计与调试；参与分片流式传输、HTTP Range 断点续传、下载日志记录、进行中下载取消等关键传输逻辑；参与 SQLite 表结构设计与持久化接口实现，包括 files、receivers、download_logs 和 download_grants 等表。

在安全与权限控制方面，杨秉熹参与 PIN 验证、接收端 token 鉴权、接收端禁用/启用、上传权限控制和开放路径过滤逻辑实现，确保列表、目录树、单文件下载和 ZIP 打包使用一

致的授权规则；同时参与 AES-CTR 可选加密和基于偏移的断点续传兼容性设计。测试方面，杨秉熹参与下载流、ZIP 流、接收端禁用、上传权限和网络工具相关测试，验证核心传输行为的正确性。非技术工作方面，杨秉熹负责演示素材整理、PPT 制作和部分报告材料组织。

9 测试、限制与改进方向

当前测试覆盖了浏览器明文下载、目录 ZIP 下载、接收端禁用阻断新下载、禁用中断进行中的下载、禁用后恢复、浏览器上传权限、上传目录配置和 PIN 相关辅助逻辑。这些测试集中验证了项目最重要的行为：鉴权、状态控制和流式下载是否按预期工作。

项目也有明确边界。第一，系统默认面向可信局域网，若暴露到公网，需要增加 HTTPS、速率限制和更严格的认证策略。第二，浏览器下载位置由浏览器决定，不能像 CLI 一样写入任意绝对路径。第三，浏览器上传接口目前会读取上传内容后写入磁盘，超大文件上传仍有优化空间。第四，SQLite 适合单机发送端，不适合多发送端并发共享同一元数据库。

10 总结

PyLocalSend 的技术重点不在于复杂协议，而在于把局域网临时文件共享中的关键问题处理清楚：发送端登记路径而不复制内容，下载时按分片流式返回；CLI 支持并行、断点续传和可选加密；WebUI 让接收端零安装使用；PIN、token、grants 和接收端状态提供足够轻量的访问控制。整体实现与项目定位一致，适合课堂演示、实验室资料分发和局域网临时协作等场景。